

1.

不論輸入是遞增還是遞減順序，HEAPSORT 的時間複雜度都是 $\Theta(n \log n)$ 。原因是：

- BUILD-MAX-HEAP() 建堆始終需要 $\Theta(n)$
- 接著執行 $n-1$ 次 HEAPIFY()，每次 $O(\log n)$ ，合計 $\Theta(n \log n)$

HEAPSORT 沒有「最佳情況」——即使輸入一開始就已排好序，它仍然會跑完整個流程。

2.

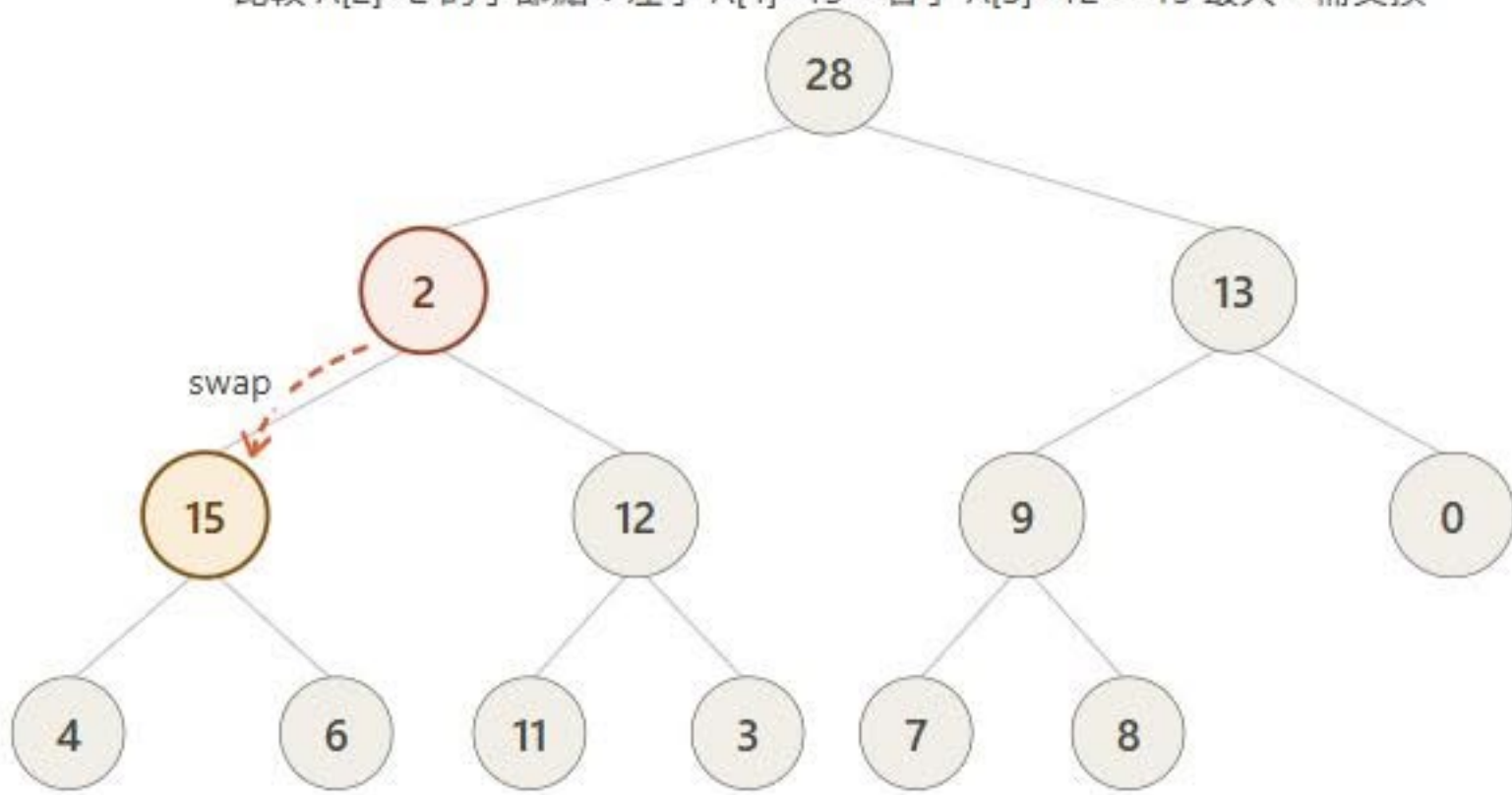
陣列 (1-indexed) : $A = \langle 28, 2, 13, 15, 12, 9, 0, 4, 6, 11, 3, 7, 8 \rangle$

- $A[2] = 2$, 其左子 $A[4] = 15$, 右子 $A[5] = 12$
- 最大值是 15 (在 $i=4$) , 與 $A[2]$ 交換 $\rightarrow 2$ 下移到 $i=4$
- 接著對 $i=4$ 繼續 : $A[4]=2$, 左子 $A[8]=4$, 右子 $A[9]=6$
- 最大值是 6 (在 $i=9$) , 與 $A[4]$ 交換 $\rightarrow 2$ 下移到 $i=9$
- $i=9$ 是葉節點 , 停止

下面是每一步的視覺化 : **共兩次交換 :**

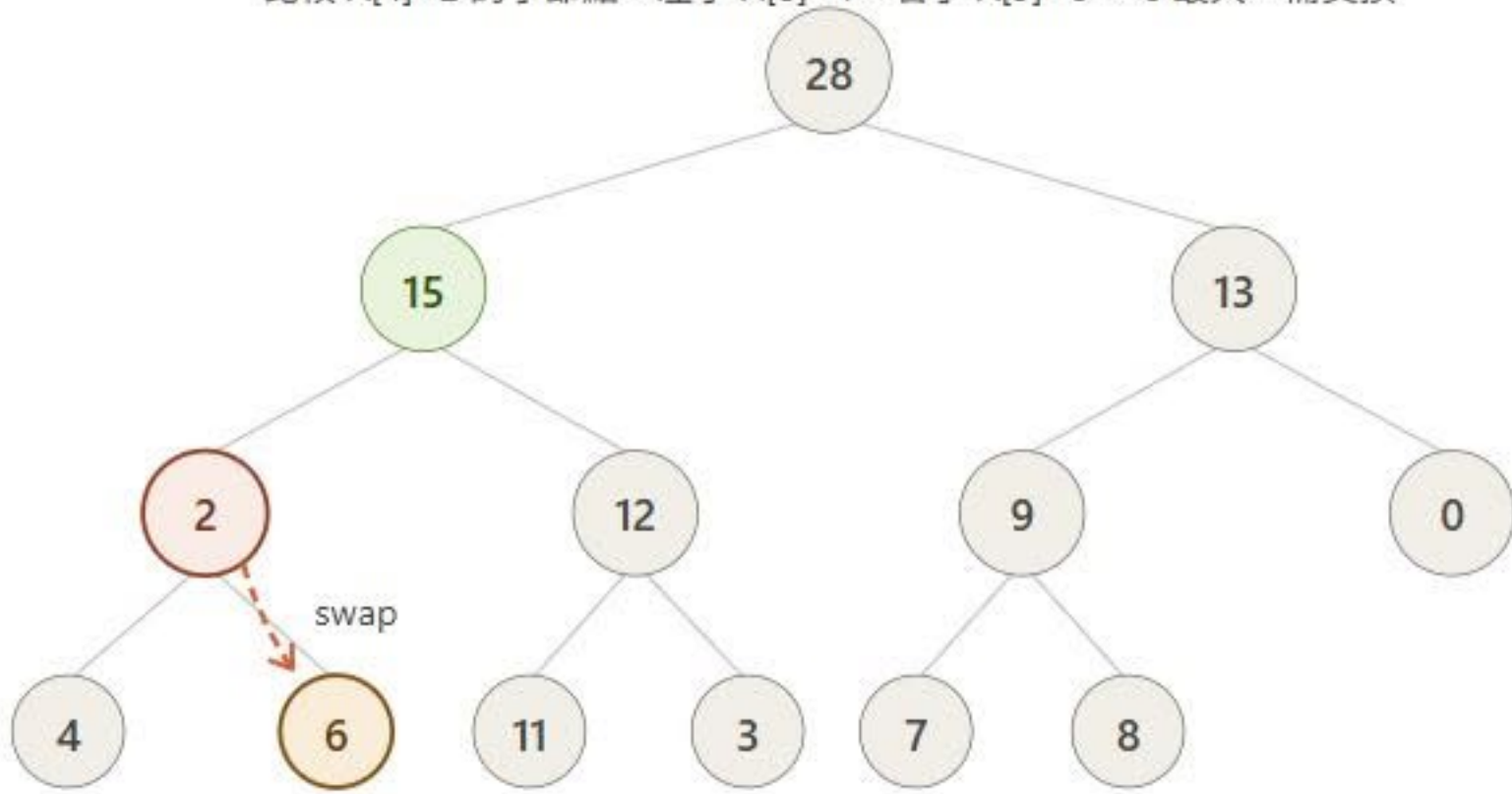
初始狀態

比較 $A[2]=2$ 的子節點：左子 $A[4]=15$ ，右子 $A[5]=12 \rightarrow 15$ 最大，需交換



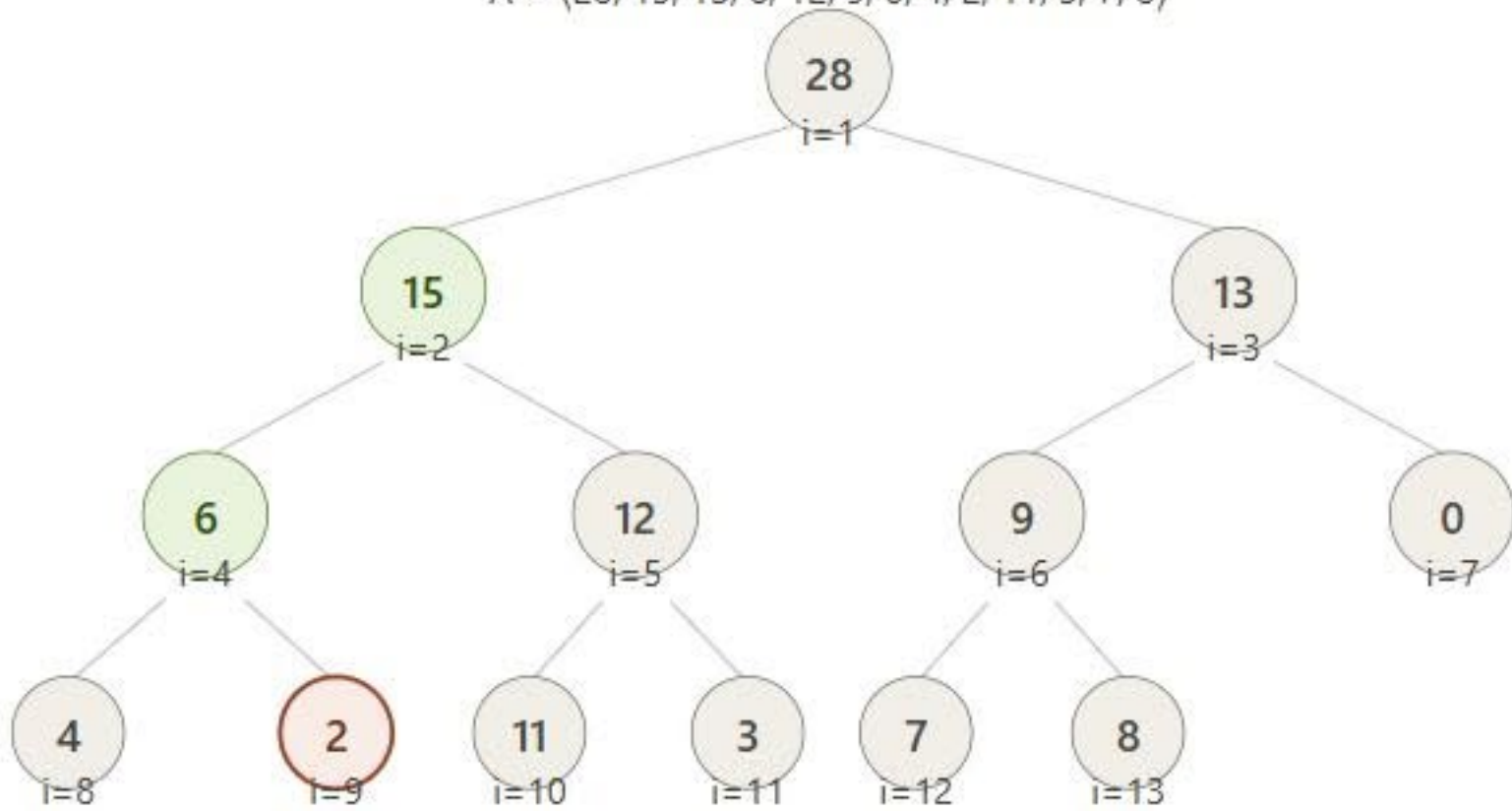
Step 1 : $\text{swap}(A[2], A[4]) \rightarrow$ 繼續對 $i=4$ 做 HEAPIFY

比較 $A[4]=2$ 的子節點：左子 $A[8]=4$ ，右子 $A[9]=6 \rightarrow 6$ 最大，需交換



Step 2 : $\text{swap}(A[4], A[9]) \rightarrow i=9$ 為葉節點，結束

$A = \langle 28, 15, 13, 6, 12, 9, 0, 4, 2, 11, 3, 7, 8 \rangle$



○ = 下沉的節點 (2)

○ = 即將被交換上去

○ = 已提升的節點

-- 以下解答也給對，但期中考若有出現相關操作會說明清楚正確應為 1 base --

陣列 (0-indexed) : $A = \langle 28, 2, 13, 15, 12, 9, 0, 4, 6, 11, 3, 7, 8 \rangle$

所以 `MAX-HEAPIFY(A, 2)` 作用在 **$A[2] = 13$** :

- $A[2]=13$ ，左子 $A[5]=9$ ，右子 $A[6]=0 \rightarrow 13$ 已經最大，不需要交換，直接結束。

這是因為 $13 > 9$ 且 $13 > 0$ ，heap 性質已滿足。